

Microservices Communication Patterns - API Gateways

Introduction

Il problema ereditato

"Il nostro problema era la comunicazione diretta client-microservizi. Porta a: complessità del client, interazioni chatty, aumento della latenza, accoppiamento stretto tra client e struttura del backend, difficoltà nel gestire le problematiche trasversali, duplicazione della logica tra diversi tipi di client."

"Se la comunicazione diretta può sembrare fattibile per un numero molto ridotto di servizi e un singolo tipo di client, diventa rapidamente ingestibile per applicazioni più grandi e complesse."

La soluzione annunciata

"Il pattern più fondamentale e ampiamente adottato per risolvere questi problemi è l'API Gateway pattern. Un API Gateway funge da punto di ingresso unico per tutte le richieste dei client ai microservizi di backend. Invece di parlare direttamente con decine di servizi, il client parla con il gateway."

Gateway Routing, Gateway Aggregation, Gateway Offloading

Prima dell'API Gateway, la lezione introduce i tre sotto-pattern che lo compongono. È importante: l'API Gateway non è un pattern monolitico, ma la combinazione di tre capacità distinte.

Pattern 1 — Gateway Routing

Definizione: instradare le richieste client in arrivo verso più microservizi di backend diversi, esponendo un unico insieme unificato di endpoint all'applicazione client.

"Il client chiama un unico endpoint del gateway, invece di dover conoscere indirizzo e porta specifici di ciascun servizio. Il gateway instrada intelligentemente la richiesta al microservizio downstream appropriato in base a percorso, metodo, header e così via."

Esempio:

Client → GET /api/products/123 → [GATEWAY] → <http://catalog-service/products/123>

Client → GET /api/orders → [GATEWAY] → <http://ordering-service/orders>

I 3 vantaggi

1. Disaccoppiamento dalla topologia del backend ★

"I client sono completamente all'oscuro di come il backend sia scomposto in microservizi o di dove risiedano fisicamente. Se rifattorizzate i microservizi — magari dividendone uno in due, o cambiando l'indirizzo interno di un servizio — il client non deve cambiare. Viene aggiornata solo la configurazione di routing nel gateway."

2. Semplificazione del client

Un unico punto di ingresso coerente per tutte le interazioni di backend.

3. API versioning e deployment blue-green ★

"È possibile instradare il traffico verso la versione 1 del servizio per alcuni utenti e la versione 2 per altri: A/B testing, canary release, o il passaggio istantaneo del traffico da una vecchia a una nuova versione durante un deployment blue-green. Il tutto gestito a livello di gateway, senza modificare il codice del client. Se una versione presenta un errore, è possibile ripristinare rapidamente la configurazione di routing."

"In sostanza, il gateway API agisce come un reverse proxy intelligente che usa il routing di livello 7 (livello applicazione)."

Definizione: routing di livello 7 — decisioni di instradamento basate sul **contenuto applicativo** della richiesta (percorso URL, header HTTP, metodo), non solo su IP e porta (livello 4). È ciò che permette al gateway di distinguere /products da /orders.

Pattern 2 — Gateway Aggregation

Definizione: il client effettua **una singola richiesta** a un endpoint del gateway, che rappresenta **un'operazione composita** che richiede dati da più servizi downstream.

Il flusso:

1. Il gateway riceve la richiesta
2. Invia **più richieste individuali** ai microservizi pertinenti, **in parallelo o in sequenza**
3. Quando tutti rispondono, **aggrega i risultati in un'unica risposta consolidata**
4. La invia al client

Esempio — la dashboard:

"La home dashboard deve mostrare informazioni utente, riepilogo ordini recenti, e raccomandazioni personalizzate."

Senza aggregazione

Il client fa **3 chiamate API separate**:
Customer, Ordering, Recommendation

Il client combina i dati

Con aggregazione

Il client fa **1 chiamata**: GET /dashboard

Il gateway chiama i 3 servizi, combina le risposte e restituisce un unico payload

I 3 vantaggi

1. **Riduce chattiness e latenza** — *"particolarmente vantaggioso su reti ad alta latenza come le connessioni mobili"*
2. **Semplifica la logica del client** — *"il client non deve più orchestrare chiamate multiple. Questa complessità è stata trasferita al gateway"*

3. **Astrazione** — il client è ulteriormente protetto dai dettagli del backend

⚠ L'avvertenza

*"È importante garantire che **il gateway non diventi esso stesso un collo di bottiglia** quando esegue l'aggregazione, e che **gestisca con grazia i guasti dei servizi downstream**."*

Pattern 3 — Gateway Offloading

Definizione: centralizzare funzionalità comuni — i **cross-cutting concerns** — dentro il gateway, scaricando queste responsabilità dai singoli microservizi.

Le funzionalità tipicamente centralizzate:

Concern	Cosa fa
SSL Termination	Gestire HTTPS e decifrare il traffico
Authentication / Authorization	Verificare l'identità degli utenti e controllare i permessi
Caching	Memorizzare le risposte comuni per migliorare le prestazioni
Rate limiting / Throttling	Proteggere i servizi di backend dal sovraccarico
Logging, tracing, correlation	Per l'osservabilità
IP allow-list / deny-list	Filtrare il traffico per origine

*"Invece di implementare tutti questi aspetti **in ogni singolo microservizio**, possiamo scaricarli nel gateway."*

I 4 vantaggi

1. **Semplifica lo sviluppo dei microservizi** — *"i team possono concentrarsi **esclusivamente sulla logica di business**. Non devono preoccuparsi di implementare autenticazione complessa o codice di rate limiting"*
2. **Uniformità** — *"garantisce che questi aspetti critici siano implementati **in modo uniforme e corretto su tutte le richieste**"*
3. **Riduce la duplicazione del codice** — *"evitando di reimplementare questi concern **in decine di microservizi**"*
4. **Centralizza la gestione** — aggiornare e configurare in **un unico luogo**

Riepilogo dei tre pattern

Pattern	A cosa serve
Routing	Indirizzare il traffico al servizio corretto, disaccoppiando il client dalla topologia
Aggregation	Ridurre la chattiness consolidando più chiamate backend
Offloading	Centralizzare funzionalità comuni e semplificare i microservizi

API Gateway Pattern

Definizione

Un API Gateway è un server che funge da punto di ingresso unico per tutte le richieste dei client verso i microservizi di backend.

"Si colloca **tra i client e i microservizi** e può: **instradare** le richieste ai microservizi corretti, **aggregare** i dati di più microservizi in un'unica risposta, e gestire **aspetti trasversali essenziali** come autenticazione, autorizzazione, SSL termination, rate limiting e logging."

★ L'analogia con il Facade pattern

"Si può pensare che sia **simile al pattern Facade** della progettazione orientata agli oggetti: **fornisce un'interfaccia semplificata e unificata che incapsula il più complesso sistema sottostante** di microservizi."

"Il gateway API agisce come **un reverse proxy, ma con intelligenza aggiuntiva e funzionalità specifiche per l'ambiente dei microservizi.**"

Le 4 capacità (i sotto-pattern in azione)

1. **Reverse proxy / Gateway Routing** — disaccoppia i client dalla struttura del backend
2. **Request Aggregation** — riduce la comunicazione chatty
3. **Cross-cutting concerns** — auth, rate limiting, SSL, logging centralizzati
4. **Protocol translation** ★

"Il gateway **accoglie richieste API RESTful dal client e le traduce in comunicazioni gRPC** per i microservizi interni."

Questa è la soluzione elegante al trade-off della Lezione 7: il browser parla REST (che capisce), i servizi interni parlano gRPC (veloce), e **il gateway fa da traduttore.**

★ Il flusso completo di una richiesta

1. Client invia richiesta all'endpoint pubblico del **gateway** (REST o GraphQL)

↓

2. **PRE-PROCESSING** (cross-cutting concerns)

- valida la richiesta
- controlla IP allow-list
- gestisce autenticazione/autorizzazione
(spesso validando il token presso l'Identity Service)
- controlla il rate limiting

↓

3. **ROUTING**

- in base a percorso, header o altri criteri,
determina quale microservizio deve gestire la richiesta

↓

4. **PROTOCOL TRANSLATION** (se necessario)

- es. converte HTTP in chiamate gRPC interne

↓

5. FORWARD ai microservizi di destinazione

↓

6. AGGREGATION (se la richiesta richiedeva dati da più fonti)

↓

7. TRANSFORMATION della risposta + invio al client

"Durante l'intero processo, il gateway è anche responsabile di **logging, monitoring**, e dell'applicazione di **pattern di resilienza come retry o circuit breaker** durante la comunicazione con i microservizi downstream."

Design: Microservices Architecture with API Gateway

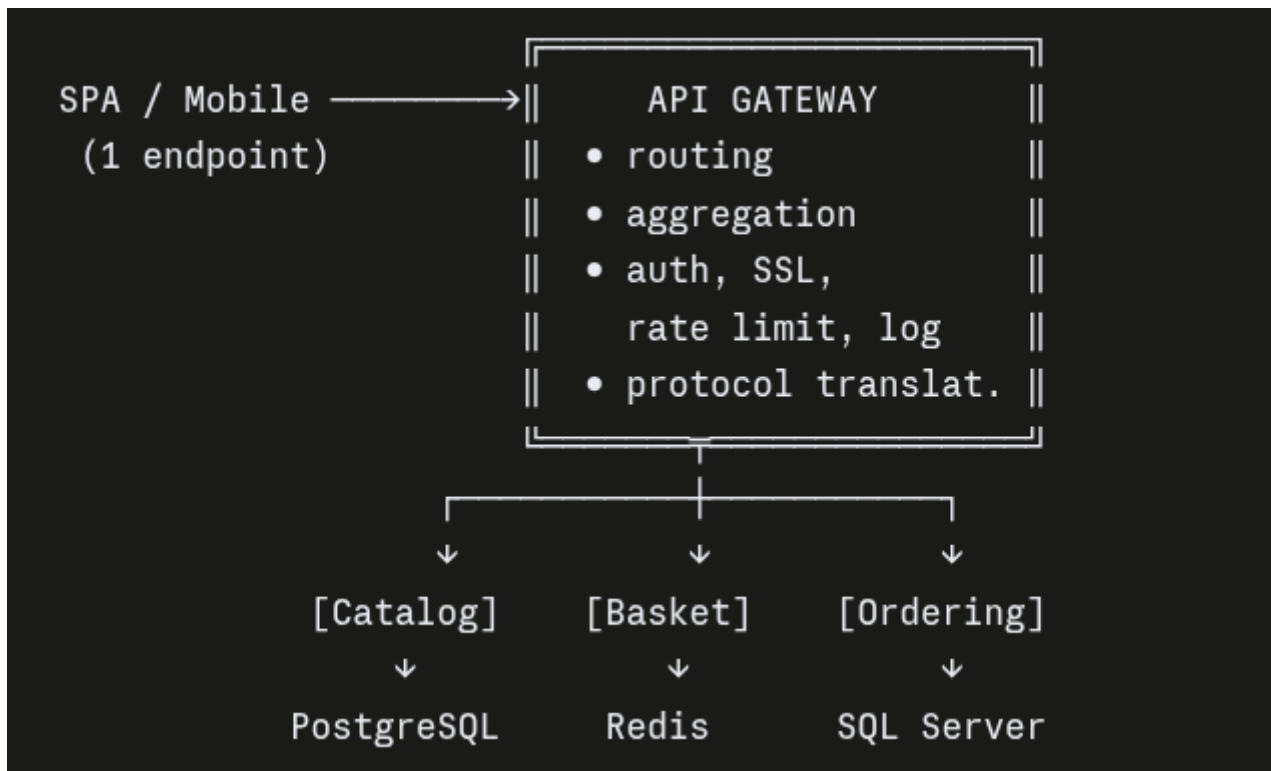
Toolkit aggiornato

Nuovi requisiti non funzionali:

- **Disaccoppiare il client dalla topologia** del backend
- **Centralizzare i cross-cutting concerns** (auth, autorizzazione, rate limiting)
- **Facilitare il recupero efficiente dei dati** tramite aggregazione al gateway

Nuovo pattern: API Gateway, che sfrutta intrinsecamente **gateway routing, aggregation e offloading**.

Il design



Cosa cambia:

"Tutte le richieste dalle applicazioni client sono ora dirette a un unico endpoint ben noto: l'endpoint dell'API Gateway. Il client non ha più bisogno di conoscere l'indirizzo dei singoli microservizi."

Effetto sui microservizi:

*"I nostri singoli microservizi ora ricevono **principalmente richieste dal gateway**. Possono essere **più semplici**, perché molti concern trasversali sono gestiti dal gateway, e si **concentrano sulla logica di business e sulla gestione dei dati**."*

Le 3 categorie di API Gateway

Categoria	Esempi	Note
Open source / proxy	nginx, Envoy	"Spesso usati come base per gateway personalizzati o come parte di soluzioni service mesh "
Cloud gestiti	Amazon API Gateway (AWS), Azure API Management, Google Cloud API Gateway	"Soluzioni robuste e gestite"
Framework / librerie	Spring Cloud Gateway (Java), Ocelot e YARP (.NET)	"Consentono di creare applicazioni gateway personalizzate"
<i>"La scelta dipende dalle esigenze specifiche, dall'infrastruttura esistente e dalle competenze del team."</i>		

★ I 5 svantaggi dell'API Gateway

Questa parte è importante quanto i vantaggi.

1. Può diventare un collo di bottiglia

"Tutto il traffico client passa attraverso il gateway, quindi deve essere altamente performante e scalabile. Se non progettato correttamente, diventa il nuovo collo di bottiglia."

2. Rischio di Single Point of Failure ★

"Se il gateway si guasta, tutti i client perdono l'accesso ai servizi di backend. Il gateway stesso deve essere distribuito in configurazione ad alta disponibilità — ad esempio con istanze multiple e load balancing."

Nota l'ironia: abbiamo passato la Lezione 4 a eliminare il single point of failure, e ora ne introduciamo uno nuovo. La soluzione è la stessa: **replicare**.

3. Overhead di sviluppo e gestione

È un altro pezzo di infrastruttura da sviluppare/configurare, distribuire, aggiornare e monitorare.

4. ★ Rischio del "God Gateway" — un anti-pattern

"C'è la tentazione di inserire troppa logica di business o orchestrazione complessa nel gateway. Se ciò accade, il gateway stesso può diventare un'applicazione complessa e monolitica, annullando molti vantaggi. Dovrebbe concentrarsi principalmente sulle responsabilità specifiche del gateway."

Questo è il modo più comune in cui l'API Gateway fallisce: diventa il nuovo monolite, solo con un nome diverso.

5. Aumento della latenza

"Mentre l'aggregazione riduce la latenza percepita dal client, ogni richiesta ha ora almeno un hop di rete in più attraverso il gateway."

Code: Microservices Architecture with API Gateway Design

Cartella 8-eshop-microservices-yarp-api-gw. Il gateway è implementato con **YARP** (Yet Another Reverse Proxy), la libreria reverse-proxy di Microsoft per .NET.

La nuova struttura

```

eshop-microservices/
├─ AppHost/
├─ Frontend/
│   └─ WebApp/
├─ Gateways/           ← NUOVA CARTELLA
│   └─ YarpApiGateway/ ← il gateway come microservizio separato
└─ Services/
    ├─ Catalog/
    ├─ Basket/
    └─ Ordering/

```

★ La modifica chiave nell'AppHost

// Il gateway referencia TUTTI i microservizi downstream

```

var gateway = builder.AddProject<YarpApiGateway>("apigateway")
    .WithReference(catalog)
    .WithReference(basket)
    .WithReference(ordering);

```

// La WebApp referencia SOLO il gateway

```

builder.AddProject<WebApp>("webapp")
    .WithReference(gateway); // ← e basta!

```

"Il gateway fa riferimento a tutti i microservizi downstream, perché deve conoscere i loro nomi di service discovery per instradare il traffico verso di essi."

"Ma soprattutto, l'applicazione web ora fa riferimento SOLO al gateway API, e non più ai microservizi interni. Non ha più un riferimento diretto a Catalog, Basket e Ordering. Tutte le comunicazioni con il backend passeranno attraverso il gateway."

Questo è il cambiamento architetturale visibile in una riga di configurazione.

Lato WebApp: da 3 HttpClient a 1

// PRIMA (Lezione 7):

```

AddHttpClient<CatalogApiClient>(c => c.BaseAddress = new("https+http://catalog"));
AddHttpClient<BasketApiClient>(c => c.BaseAddress = new("https+http://basket"));
AddHttpClient<OrderingApiClient>(c => c.BaseAddress = new("https+http://ordering"));

```

// ORA:

```

AddHttpClient<ApiClient>(c => c.BaseAddress = new("https+http://apigateway"));

```

E nella classe client, gli URL cambiano forma:

GetProducts() → GET /catalog-service/products

GetBasket() → GET /basket-service/basket/{username}

UpdateBasket() → POST /basket-service/basket

*"Il gateway capisce, **dal prefisso del percorso**, che queste richieste vanno instradate ai microservizi Catalog / Basket."*

Il progetto YARP Gateway

Program.cs:

```
builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"));

builder.Services.AddRateLimiter(...); // gateway offloading

var app = builder.Build();
app.MapReverseProxy();
```

appsettings.json — la configurazione dichiarativa di YARP:

```
"ReverseProxy": {
  "Routes": {
    "catalog-route": {
      "ClusterId": "catalog-cluster",
      "Match": { "Path": "/catalog-service/{**catch-all}" },
      "Transforms": [ { "PathRemovePrefix": "/catalog-service" } ]
    },
    "ordering-route": {
      "ClusterId": "ordering-cluster",
      "Match": { "Path": "/ordering-service/{**catch-all}" },
      "RateLimiterPolicy": "fixed", // ← GATEWAY OFFLOADING
      "Transforms": [ { "PathRemovePrefix": "/ordering-service" } ]
    }
  },
  "Clusters": {
    "catalog-cluster": {
```

```

"Destinations": {
  "destination1": { "Address": "https+http://catalog" } // ← service discovery
}
}
}
}

```

I due concetti di YARP:

Concetto	Cosa fa
Route	<i>"Definisce come le richieste in arrivo al gateway vengono soddisfatte". Il Match.Path decide quali richieste catturare; il ClusterId dice dove mandarle</i>
Cluster	<i>"Definisce una o più destinazioni verso i microservizi di backend"</i>

Le Transforms:

*"Queste trasformazioni spesso **eliminano il prefisso** prima di inoltrare la richiesta al backend."*

Cioè: il client chiama /catalog-service/products, ma il microservizio Catalog espone /products. La transform rimuove il prefisso. **Il prefisso serve solo al gateway per il routing.**

Il service discovery:

*"L'indirizzo di destinazione è https+http://catalog, e il nome **catalog** viene risolto tramite il service discovery a un'istanza effettivamente in esecuzione del microservizio. **Questo è il gateway routing in azione.**"*

Cosa si osserva eseguendo

Nuove risorse nella dashboard: oltre a backing services, i 3 microservizi e la webapp, compare **apigateway**.

Variabili d'ambiente:

- Il **gateway** ha gli indirizzi di service discovery di **tutti** i microservizi interni
- La **webapp** ha **solo l'endpoint del gateway** ★

Il graph view — "particolarmente perspicace":

*"Mostra chiaramente che **la webapp effettua chiamate solo al gateway**, e il gateway effettua chiamate ai microservizi interni. **Questa immagine conferma il ruolo del gateway come intermediario centrale.**"*

Le tracce end-to-end

Caso 1 — Lista prodotti:

```
WebApp → GET /catalog-service/products → [API GATEWAY]
                                         ↓ (routing + transform)
                                         Catalog Service → GET /products
                                         ↓
                                         PostgreSQL (SELECT)
```

Caso 2 — Aggiungi al carrello — "tracce molto lunghe":

```
1. WebApp → GET /basket-service/basket/swn → [GATEWAY] → Basket → legge da Redis
2. WebApp → POST /basket-service/basket → [GATEWAY] → Basket
                                         ↓ (chiamata sincrona della Lez. 7)
                                         Catalog → GET /products/{id} × N
                                         ↓
                                         PostgreSQL
                                         ↓
                                         Basket scrive in Redis
```

Nota: la chiamata sincrona Basket → Catalog della Lezione 7 **non passa dal gateway**. Il gateway gestisce il traffico **nord-sud** (client ↔ backend); la comunicazione **est-ovest** (servizio ↔ servizio) resta diretta. È una distinzione architetturale importante.

📌 **Suggerimento pratico dalla lezione:**

*"Questo tipo di tracce lunghe... possiamo usare **"Explain traces"** di GitHub Copilot Chat. Sarà un'ottima spiegazione per ogni fase del tracing."*

Problem: Client Apps have Different UI Requirements

Lo scenario

*"L'e-shop sta espandendo la sua portata. Abbiamo l'applicazione web, a cui gli utenti accedono tramite browser — **questa piattaforma può visualizzare informazioni ricche e dettagliate**. Ma ora vogliamo lanciare anche **un'applicazione mobile nativa per iOS e Android**."*

I vincoli diversi del mobile:

- **Schermi più piccoli** → le UI devono essere più concise
- **Connessioni dati lente o mobili** → è importante **minimizzare il trasferimento dei dati**
- I dati necessari per una vista mobile possono essere **un sottoinsieme, o una vista aggregata diversa** rispetto al web

★ Il problema delle API "one size fits all"

*"Il microservizio Product potrebbe restituire **50 campi** per un prodotto. Il client **web** ne visualizza **40** in una vista dettagliata. Il client **mobile**, invece, ha bisogno **solo di 10**"*

campi chiave per una vista sintetica — gli altri 40 campi sono overhead, che sprecono banda ed elaborazione sul dispositivo mobile.

I due sintomi:

Sintomo	Descrizione
Over-fetching	Il servizio restituisce troppi dati; il mobile spreca banda e batteria
Under-fetching + chattiness	Il client deve combinare dati da più chiamate API generiche per costruire una singola schermata

*"Questa logica di manipolazione dei dati lato client rende l'applicazione **più complessa, più difficile da mantenere e potenzialmente più lenta**, in quanto svolge un lavoro **che potrebbe essere gestito meglio vicino al backend**."*

*"In sostanza, **le API one-size-fits-all dei nostri microservizi core spesso non si adattano perfettamente a nessun client specifico**."*

Cosa serve

1. **Fornire a ciascun tipo di client un'API e payload specificamente adattati** ai suoi requisiti di UI
2. **Spostare la logica di aggregazione e formattazione specifica del client a un componente backend dedicato**

Backends for Frontends Pattern (BFF)

Definizione

Il pattern BFF prevede la creazione di un servizio backend (o un'istanza di API Gateway) dedicata e separata per ogni frontend, applicazione o esperienza utente.

*"Invece di avere **un gateway API generico che cerca di servire tutti i client**, avete **diversi gateway più mirati**."*

```
Web SPA      → [Web BFF]      ┌┐
iOS App      → [Mobile BFF]  ┌┐→ [Catalog] [Basket] [Ordering] [Discount]
Partner API  → [Partner BFF] ┌┐      (microservizi generici)
```

*"Ogni BFF è responsabile di fornire **un'API perfettamente adattata alle esigenze di quel specifico frontend**. Comunica con i microservizi generici downstream, ma **agisce da intermediario, modellando dati e interazioni specificamente per il suo frontend**."*

I 4 motivi per usarlo

1. **Requisiti di dati significativamente diversi tra client** ★ — "il driver principale"

*"Il client web ha bisogno di **informazioni ricche e dettagliate** per il desktop. Il mobile ha bisogno di **una versione sintetizzata e leggera**, ottimizzata per schermi piccoli e reti lente. **Un Web BFF e un Mobile BFF possono interrogare gli stessi microservizi e adattare perfettamente la risposta ai rispettivi frontend**."*

2. Ridurre la logica lato client

*"Se i client eseguono molte manipolazioni di dati, aggregazioni o formattazioni complesse, **questa logica può essere spostata nei rispettivi BFF**. Questo rende l'applicazione client **più sottile e semplice**."*

3. Evoluzione indipendente ★

*"Il team che lavora sull'app mobile può evolvere la propria API mobile per supportare nuove funzionalità mobili senza impattare il web BFF o i servizi core. Questo consente **cicli di rilascio indipendenti più rapidi** per i diversi frontend."*

Questo è il vantaggio più sottile ma forse il più prezioso: **il BFF appartiene al team del frontend**, non al team backend.

4. Evitare il "God Gateway"

*"Se un singolo gateway API tenta di soddisfare le esigenze uniche di molti client diversi, **può diventare incredibilmente complesso, gonfio, e un collo di bottiglia**. Il **BFF aiuta a dividere questa complessità lungo i confini dei client**."*

L'esempio dell'e-shop

Web App BFF: quando la SPA deve mostrare una lista prodotti, chiama un endpoint sul Web BFF → questo chiama **Catalog** e forse **Discount**, aggrega, e **formatta in modo ottimale per la visualizzazione web ricca**.

Mobile App BFF: quando l'app mobile deve mostrare la stessa lista, chiama il Mobile BFF → questo chiama **gli stessi Catalog e Discount**, ma **restituisce un payload più conciso, magari con URL di immagini più piccoli o meno dettagli, ottimizzato per l'esperienza mobile**.

*"Entrambi i BFF interagiscono con **lo stesso insieme di microservizi di backend**, ma **agiscono come traduttori o aggregatori**, adattando l'esperienza alle loro specifiche applicazioni frontend."*

Design: Microservices Architecture with BFF Pattern

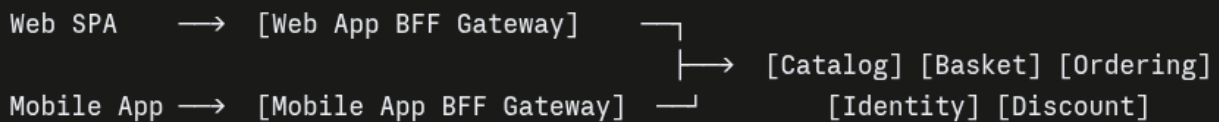
Toolkit aggiornato

Nuovi requisiti non funzionali:

- **API ottimizzate per ogni tipo di client** (web e mobile), con payload e pattern di interazione adattati
- **Evoluzione indipendente della logica di backend** che serve ogni specifico client

Nuovo pattern: Backend For Frontend.

Il design



"Invece di **un singolo gateway API generico**, ora abbiamo **due istanze distinte di gateway API**, ognuna delle quali funge da backend per un frontend."

★ Scelte tecnologiche — il poliglottismo dei BFF

BFF	Possibile stack	Ottimizzazioni
Web App BFF	ASP.NET Core Web API	Endpoint RESTful o uno schema GraphQL adattato all'esperienza web
Mobile App BFF	ASP.NET Core, o Node.js	Compressione della risposta, payload JSON molto snelli, o un endpoint GraphQL progettato per le esigenze mobili

"Ognuno può essere sviluppato usando lo stack tecnologico che meglio si adatta alle competenze del team frontend. Ciò significa che possiamo usare sviluppo poliglotta con diversi livelli BFF."

Nota il punto: se il team mobile è composto da sviluppatori JavaScript, il Mobile BFF può essere in Node.js — **perché appartiene a loro**.

I 4 svantaggi

1. Aumento del numero di unità deployabili

"Ogni BFF è un nuovo servizio backend: più componenti da sviluppare, testare, distribuire, monitorare e scalare."

2. Duplicazione del codice tra BFF ★

"Se il Web BFF e il Mobile BFF devono entrambi chiamare lo stesso insieme di microservizi ed eseguire un'aggregazione simile, c'è il rischio di duplicare parte della logica di orchestrazione. Questa situazione va gestita, forse attraverso librerie client condivise che i BFF possono usare per interagire con i servizi core."

Nota il trade-off ricorsivo: il BFF risolve la duplicazione **tra client**, ma può introdurre duplicazione **tra BFF**.

3. Un hop di rete in più

"Il BFF introduce un ulteriore nodo di rete nella catena. Mentre riduce la chattiness complessiva del client, il BFF stesso deve essere altamente performante."

4. Complessità operativa

"Più servizi significa più parti mobili nei vostri sistemi."

"Per applicazioni con esperienze utente distinte tra diversi tipi di frontend, i vantaggi dei BFF spesso superano la complessità aggiunta. Come sempre, è un trade-off, e come architetti dobbiamo soppesare vantaggi e svantaggi."

Service-to-Service Communications between Backend Internal Microservices

Questa sezione sposta il focus dal traffico **client** ↔ **backend** al traffico **backend** ↔ **backend**.

La best practice di partenza

"La best practice è ridurre il più possibile la comunicazione tra i servizi. Tuttavia, in alcuni casi non possiamo ridurla, a causa dei requisiti del client o dell'operazione richiesta."

Il caso semplice: 2 chiamate

Scenario: l'utente aggiunge un articolo al carrello, ma l'operazione deve ottenere dati da **Product** e **Pricing**.

```
Client → [API Gateway] → [Basket] —sync→ [Product]
                                   —sync→ [Pricing]
```

I problemi già presenti:

*"Queste chiamate interne fanno accoppiare i microservizi. Diventano **dipendenti** l'uno dall'altro. Se uno dei microservizi è fermo, non può restituire i dati al client. Non c'è tolleranza ai guasti."*

"Crea dipendenze dirette a runtime. Se il servizio Catalog o il servizio Pricing sono lenti o non disponibili, il microservizio Basket si blocca e non può completare il suo compito, causando potenzialmente il fallimento dell'intera richiesta dell'utente."

"In questo caso abbiamo solo due chiamate sincrone interne, quindi può essere relativamente gestibile e accettabile per alcuni sistemi."

★ Il caso grave: la catena profonda

Scenario — Order Fulfillment:

```
Client → [Gateway] → [Ordering] → [Payment]      (autorizza il pagamento)
                                   → [Inventory]   (riserva le scorte)
                                   → [Shipping]    (organizza la spedizione)
                                   → [Notification] (invia email di conferma)
```

"Come si può notare, ci sono sei richieste HTTP sincrone concatenate per una singola richiesta HTTP del client."

"Se tutti questi passaggi sono implementati come chiamate sincrone dirette, dove ogni servizio attende il successivo, si incorre in seri problemi. È ovvio che questa comunicazione aumenta la latenza e influisce negativamente su prestazioni, scalabilità e disponibilità."

Le domande critiche:

"Cosa succede se il passo 5 o 6 fallisce? O se alcuni servizi non funzionano? Anche se non sono down, potrebbero essere occupati e non rispondere in modo tempestivo, causando latenze elevate non accettabili."

*"Questo tipo di comunicazione sincrona e profondamente concatenata **annulla molti dei vantaggi che cerchiamo di ottenere dai microservizi.**"*

Perché la latenza si somma in modo devastante: se ogni chiamata impiega 100ms, 6 chiamate in catena = **600ms minimo**. E la **disponibilità si moltiplica**: 6 servizi al 99% di uptime ciascuno = $0.99^6 \approx 94\%$ di disponibilità complessiva. Hai peggiorato l'affidabilità aggiungendo servizi.

Problem: Service-to-Service Communications Chain Queries

Lo scenario di query

*"Un utente vuole visualizzare **i dettagli di uno dei suoi ordini passati**. La pagina richiede:"*

Dato	Da quale servizio
Info base ordine (data, importo totale)	Ordering
Indirizzo di spedizione del cliente	Customer
Lista prodotti con nomi e immagini	Catalog
Stato di spedizione attuale	Shipping

*"Tutte queste informazioni **collegate in un'unica vista concettuale dell'ordine**. Ma i **dati grezzi risiedono in diversi microservizi**, ognuno dei quali possiede un dominio di dati specifico."*

Il pattern delle "chained synchronous queries"

1. GET ordine da Ordering
2. usa il customerId → GET cliente da Customer
3. usa i productId delle righe → GET prodotto da Catalog (× N)
4. GET stato spedizione da Shipping

*"Questa sequenza di **chiamate di rete individuali e bloccanti** crea il nostro problema: **recupero inefficiente dei dati.**"*

I 4 problemi

Problema	Descrizione
Aumento della latenza	<i>"I round-trip multipli si sommano, rendendo il caricamento della pagina lento"</i>
Riduzione della disponibilità ★	<i>"Il successo dell'operazione dipende ora dalla disponibilità e reattività di TUTTI questi microservizi. Un guasto o rallentamento in uno qualsiasi dei servizi della catena può causare il fallimento dell'intera vista"</i>

Problema	Descrizione
Accoppiamento stretto	"Il client o servizio chiamante diventa implicitamente accoppiato alla distribuzione dei dati di questi molteplici microservizi downstream"
Spreco di risorse	"Più servizi consumano risorse per quello che concettualmente è un singolo compito di aggregazione dati "

Cosa serve

1. Consolidare più operazioni di recupero dati in un numero minore di interazioni
2. Astrarre la complessità della distribuzione dei dati tra microservizi
3. Fornire un unico punto per assemblare la vista composita

Service Aggregator, Service Registry/Discovery Patterns

Pattern: Service Aggregator

Definizione: introdurre un microservizio specializzato (l'Aggregator), oppure potenziare l'API Gateway con capacità di aggregazione.

Il flusso:

1. L'aggregator riceve **una singola richiesta composita** dal client o dal gateway (es. `GET /order-details/{id}` — dettagli ordine con info cliente e prodotto)
2. **Invia internamente più richieste** ai microservizi downstream pertinenti (Ordering, Customer, Catalog...)
3. **Attende le risposte**
4. **Combina/aggrega i risultati** in un'unica struttura dati coerente
5. **Invia una singola risposta aggregata** al richiedente

"Il client dell'applicazione web effettua **una sola chiamata** a questo aggregator."

I 3 vantaggi:

1. **Riduce la chattiness** e l'overhead di comunicazione
2. **Centralizza la logica** di orchestrazione complessa in un componente dedicato
3. **Semplifica il client** e astrae le interazioni con i microservizi sottostanti

Aggregator vs Gateway Aggregation — la differenza: sono lo stesso concetto applicato in due luoghi. Il **Gateway Aggregation** mette l'aggregazione **dentro il gateway** (rischio: God Gateway). Il **Service Aggregator** la mette in **un microservizio dedicato** (vantaggio: il gateway resta magro, l'aggregator scala indipendentemente).

★ Pattern: Service Registry & Discovery

Il problema:

"I nostri microservizi, **soprattutto se scalati orizzontalmente**, avranno **istanze multiple**, che possono essere **distribuite, scalate, ridotte o riavviate in modo indipendente**. Ciò significa che **le loro posizioni di rete — IP e porte — sono dinamiche e possono cambiare frequentemente**."

*"In un monolite tradizionale le posizioni dei componenti sono **statiche e note in fase di compilazione**. Ma in un ambiente cloud-native distribuito, **non possiamo hardcodare questi indirizzi**."*

Definizione: Service Registry

*"Essenzialmente **un database centralizzato** che mantiene **un elenco aggiornato di tutte le istanze di microservizi disponibili e delle loro attuali posizioni di rete (IP e porte)**."*

I due componenti

1. Service Registration

*"Quando una nuova istanza di microservizio si avvia, **si registra nel registry**, fornendo **nome, indirizzo IP e numero di porta**. Può anche fornire **informazioni sugli health check**. Quando viene spenta, dovrebbe **deregistrarsi**."*

2. Service Discovery

*"Quando un altro servizio — ad esempio il gateway — deve chiamare il microservizio Product, **non conosce direttamente IP e porta**. Invece, **interroga il registry**: "dove posso trovare le istanze del microservizio Catalog?". **Il registry restituisce un elenco di istanze sane e disponibili**, e il client usa **il load balancing lato client** per sceglierne una."*

I due tipi di discovery

Tipo	Come funziona
Client-side discovery	Il client interroga il registry e sceglie l'istanza
Server-side discovery	Il client fa una richiesta a un router o load balancer , che a sua volta interroga il registry e inoltra la richiesta

"Il nostro API Gateway spesso svolge questo ruolo di discovery lato server."

Strumenti tradizionali: Netflix Eureka, Consul, Zookeeper.

★ **La buona notizia: oggi lo fa la piattaforma**

*"Anche se è bene comprendere il pattern sottostante, **nello sviluppo cloud-native moderno spesso non è necessario implementare un service discovery da zero né gestire strumenti complessi come Eureka o Consul**."*

Piattaforma	Come lo fa
Kubernetes ★	<i>"Quando distribuite un microservizio, riceve un nome DNS interno stabile. Altri servizi possono semplicemente usare quel nome DNS, e Kubernetes si occupa della risoluzione verso un'istanza sana, incluso il load balancing"</i>
Cloud (AWS/Azure/GCP)	Soluzioni di service discovery gestite
.NET Aspire	<i>"Gestisce egregiamente il service discovery nell'ambiente di sviluppo locale" — quando un progetto referencia un altro, risolve i nomi come https+http://catalog all'istanza e porta corrette</i>

"Sebbene il pattern sia fondamentale, i dettagli di implementazione sono spesso gestiti dalla piattaforma sottostante. Ma è comunque importante sapere cosa succede sotto il cofano."

Design: Microservices Architecture with Service Aggregator, Registry/Discovery

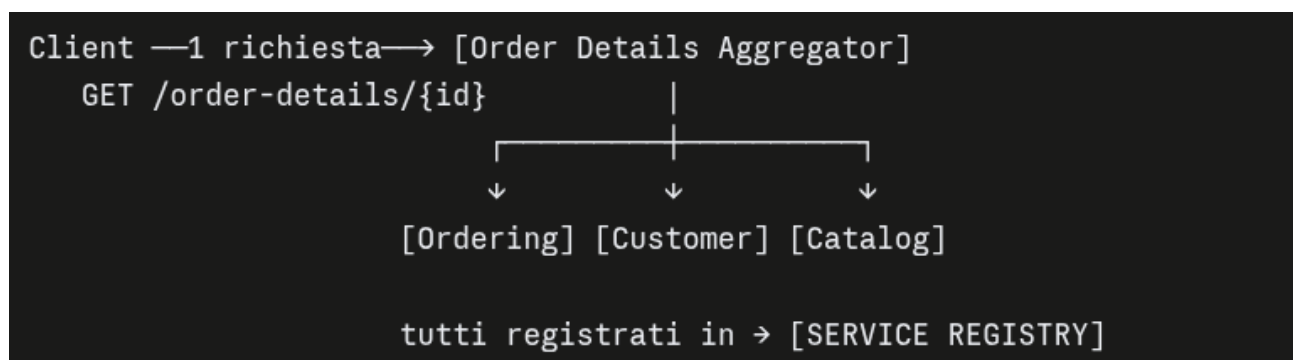
Toolkit aggiornato

Nuovi requisiti non funzionali:

- **Recupero efficiente di dati compositi** per viste che si estendono su più microservizi
- **Risoluzione dinamica della posizione dei servizi**

Nuovi pattern: **Service Aggregator** e **Service Registry/Discovery** "anche se la sua implementazione è spesso gestita dalle piattaforme sottostanti".

Il design



*"Invece di effettuare più chiamate, il client effettua una singola richiesta al microservizio **Aggregator**. L'aggregator invia richieste multiple e recupera i dati da Ordering, Customer e Catalog, **combina i risultati in un'unica risposta JSON ottimizzata** e la invia al client."*

"Come fa l'Order Details Aggregator a conoscere la posizione di rete dei microservizi downstream, soprattutto quando queste istanze possono scalare e cambiare dinamicamente? È qui che il Service Registry/Discovery è essenziale."

Scelte tecnologiche

Componente	Opzioni
Aggregator	"Un altro microservizio leggero" — ASP.NET Core Web API o stack Java. Internamente usa HttpClientFactory per chiamate REST o client gRPC generati
Service Registry	.NET Aspire in locale; Kubernetes DNS in produzione; Consul/Eureka come alternative — "ma il loro uso esplicito sta diventando meno comune, perché ora potenti orchestratori come Kubernetes gestiscono da soli il service discovery"

I 3 svantaggi

1. **L'aggregator può diventare un nuovo collo di bottiglia** — "se svolge un lavoro eccessivo o non viene scalato correttamente"
2. **Un altro servizio da sviluppare, distribuire e gestire**

3. Rischio di duplicazione — "se si finisce per creare **molti aggregator diversi che eseguono chiamate downstream simili**. Un buon design è fondamentale"

Per il service discovery:

*"Se doveste implementare e gestire il vostro service discovery — **cosa rara al giorno d'oggi** — sarebbe **un'altra parte critica dell'infrastruttura**, con le sue esigenze di disponibilità e scalabilità. **Affidarsi al discovery fornito dalla piattaforma attenua notevolmente questo problema.**"*

"I vantaggi dell'aggregator sono solitamente superiori all'overhead, e il service discovery è semplicemente un must — sia esplicito, sia fornito dalla piattaforma."

Problem: Long Running Operations Can't Be Handled with Sync Communication

Il punto chiave: l'aggregator non risolve tutto

Questa è l'osservazione più importante della sezione:

*"Il problema della catena sincrona è già stato risolto con il pattern Service Aggregator... ma l'aggregator è utile solo se il client ha bisogno di tutti questi dati in una volta sola, cosa che **NON** avviene nell'invio di un ordine. Quindi **non** risolve il problema di come questi servizi di backend collaborino efficientemente per un'operazione di tipo **COMANDO**."*

*"Il Service Aggregator è utile per operazioni a intensità di **LETTURA** — come ottenere i dettagli di un ordine con dati cliente e prodotto."*

La distinzione fondamentale:

	Query (lettura)	Command (scrittura)
Esempio	"Mostrami i dettagli dell'ordine 22"	"Piazza questo ordine"
Il client vuole	Tutti i dati, subito, in una risposta	Una conferma che l'operazione è iniziata
Soluzione	✅ Service Aggregator	❌ l'aggregator non aiuta
Il problema definito		

"Le operazioni di lunga durata non possono essere gestite con comunicazione sincrona. La comunicazione sincrona fallisce per i processi di backend complessi, e blocca le richieste degli utenti."

I 4 problemi

1. Aumento della latenza

*"Ogni chiamata sincrona della catena **aggiunge latenza di rete e tempo di elaborazione**, e l'utente finale rimane bloccato in attesa del completamento dell'intera catena. Questo porta a **tempi di risposta molto lunghi e un'esperienza utente frustrante.**"*

2. Fallimenti a cascata ★

*"Il successo dell'intera operazione dipende dalla disponibilità e rapidità di **ogni singolo servizio della catena**. Se anche un solo servizio — ad esempio al passo 5 o 6 — subisce un rallentamento o un errore, l'intera transazione di ordine può fallire. Questo è noto come **cascading failure**."*

3. Accoppiamento stretto e temporale ★

*"I servizi della catena diventano **strettamente e temporalmente accoppiati**."*

Definizione: accoppiamento temporale (temporal coupling) — due servizi devono essere **attivi nello stesso momento** perché l'operazione riesca. È una forma di accoppiamento distinta da quella sui dati o sull'API, ed è **esattamente ciò che l'asincrono elimina**.

4. Consumo di risorse

*"Il servizio chiamante **mantiene occupate le risorse** — thread e connessioni — in attesa delle risposte downstream. Durante i picchi, questo può esaurire le risorse e portare a rallentamenti o crash del sistema."*

Le 3 best practice necessarie

1. Disaccoppiamento

*"I servizi coinvolti in flussi di lavoro multi-step, come l'order fulfillment, **dovrebbero poter operare in modo indipendente, senza che altri servizi debbano essere disponibili nello stesso momento**."*

2. Resilienza

*"Il **fallimento temporaneo di un servizio nel processo** — ad esempio il servizio di notifica — **non deve causare il fallimento dell'intera operazione di business, né bloccare l'utente a tempo indeterminato**."*

Nota quanto è sensato: se l'email di conferma non parte, **l'ordine è comunque valido**. In una catena sincrona, invece, il fallimento di Notification farebbe fallire tutto.

3. Operazioni non bloccanti ★

*"Quando un utente avvia un'operazione complessa e potenzialmente lunga, **il sistema dovrebbe idealmente riconoscere la richiesta rapidamente** — "il tuo ordine è in elaborazione" — **e poi eseguire le fasi successive in background**, senza far attendere all'utente il completamento di tutto."*

La soluzione

*"Questo ci porta direttamente a **un diverso paradigma di comunicazione: la comunicazione asincrona basata su messaggi**, e ci muove verso **un'architettura event-driven**."*

→ **Lezione 9: Asynchronous Microservices Communications**

(Ed è esattamente il contenuto che, ricordi, era finito per errore all'inizio della trascrizione della Lezione 1: publisher, subscriber, message broker, eventual consistency, idempotenza.)